

AutoBenchmark: A Closed-Loop Autonomous Benchmarking Framework for Machine Learning Research

Aditya Girish

May 25, 2025

Conceptual Research Proposal / Representative Writing Sample

Status: Conceptual Proposal — Implementation Pending Resource Acquisition

This document serves as a dated record of original ideation. The system described herein is a proposition: architecturally designed, theoretically grounded, and awaiting the compute and engineering resources required for full-scale implementation.

Abstract

We propose **AutoBenchmark**, a conceptual framework for autonomously evaluating machine learning research. AutoBenchmark continuously ingests new papers or preprints, identifies their key claims and tasks, and then executes standardised experiments to verify performance. Our design features a *six-stage pipeline* (paper parsing, task setup, data preparation, model training, evaluation, and reporting) and a *three-tier compute model* (local, cluster, cloud) to scale experiments to large models. We introduce quantitative *uncertainty and reproducibility scores* for each published result, based on code and data availability and variance across repeated trials, which aim to flag unreliable claims. We illustrate AutoBenchmark with a worked example tracing a hypothetical new image-classification model, and discuss limitations such as scope and dependence on authors’ code. The evaluation plan includes metrics (accuracy, training time, reproducibility) and compute estimates (GPU-hours and costs), together with cost tables and a timeline for next steps. We draw on authoritative sources, including studies on machine learning reproducibility, standard benchmarks such as MLPerf training and inference, and platforms for sharing machine learning results such as OpenML and Papers With Code. We conclude with a discussion of potential impact and future work.

1 Introduction

Machine learning (ML) research has accelerated dramatically, producing breakthroughs in vision, language, and other domains. However, many results are reported without rigorous independent validation. A field-wide reproducibility review found that over 70% of researchers failed to reproduce others’ ML experiments, and more than half could not reproduce their own. Similarly, a replication study found that only 63.5% of published results could be duplicated, rising to 85% only when the original authors offered assistance. These findings echo broader concerns about reproducibility across the sciences. The high variability of ML training, the complexity of modern hardware and software stacks, and practices such as selective reporting all contribute to a “reproducibility crisis” in AI.

At the same time, the ML community has begun to recognise the value of systematic evaluation. Industry-led benchmarks (e.g. MLPerf) and community platforms (e.g. OpenML, Papers With Code) have provided shared tasks and datasets to fairly compare methods. MLPerf, for example, was developed by dozens of organisations to test both training and inference, yielding hundreds of reproducible measurements. OpenML has amassed thousands of datasets and tasks, powering over 1,500 studies and directly addressing reproducibility by encoding complete experimental context. Similarly, Papers With Code (and its successor systems) linked over 18,000 papers to code and leaderboards under an open licence, making it easy to find implementations and compare results.

Despite these advances, no system currently automates the continuous benchmarking of *new* ML research at scale. Researchers must manually set up experiments for each new paper — a time-consuming process, especially for large models. AutoBenchmark aims to fill this gap. We propose a framework that autonomously transforms published claims into executable pipelines, runs training and evaluation on appropriate hardware tiers, and publishes independent results. This integrates ideas from experiment-tracking tools (MLflow, Weights & Biases, Hydra, MLXP) with community benchmarks. Crucially, AutoBenchmark assigns scores for uncertainty and reproducibility: results with missing code or insufficient hyperparameter detail, for instance, receive lower confidence. This helps stakeholders gauge which reported results are well supported.

The remainder of this proposal is organised as follows. Section 2 reviews related work on benchmarking and reproducibility. Section 3 describes our system design: the six-stage pipeline, the three-tier compute model, and the scoring mechanisms. Section 4 presents a worked example of the pipeline in action. Section 5 discusses limitations and potential failure modes. Section 6 outlines an evaluation plan, including metrics, compute, and cost estimates. Section 7 compares options for hosting the resulting report, and Section 8 concludes. Throughout, we cite authoritative studies and provide cost and calculation tables.

2 Related Work

2.1 Reproducibility Challenges in Machine Learning

Reproducibility of empirical results has received increasing attention in ML. In a report from the NeurIPS 2019 Reproducibility Program, Pineau *et al.* observed that “presented results are sound and reliable” only with difficulty, since ML experiments involve a computer designed and built by humans, so reliability ought to be easier to achieve than it often is in practice. The report cites a 2016 *Nature* survey by Baker, which found that around 70% of scientists failed to reproduce others’ experiments and more than 50% failed to reproduce their own. In ML specifically, Henderson *et al.* (2018) showed that subtle differences in implementation or random seeds can flip reported improvements in deep reinforcement learning.

More recent analyses have quantified the problem further. Raff (2019) attempted to replicate results from 255 ML papers and succeeded for only 63.5% of the reported outcomes; success jumped to 85% when the original authors offered help. This suggests that a lack of documentation or missing detail, rather than fundamental flaws, often blocks reproducibility. The NeurIPS report concludes that ML faces both a *reproduction problem* and a *reporting problem*: common issues include missing code or data, unspecified hyperparameters, and opaque metric calculations. These failures waste researcher time and raise doubts about “state-of-the-art” claims in many papers.

To address this, the community has begun requiring better disclosure: major conferences now mandate code submission or reproducibility checklists. The NeurIPS reproducibility effort included such a checklist and a community challenge, aiming to raise standards for ML research. Our work aligns with this agenda: AutoBenchmark *automatically* verifies key experiments, effectively institutionalising a reproducibility check on every new contribution.

2.2 Benchmark Suites and Platforms

Benchmarks have a long history of driving progress. In traditional computing fields, industry consortia created standard benchmarks (SPEC, TPC) that accelerated hardware and software advances. In ML, **MLPerf** has emerged as an industry-driven standard. The MLPerf Training benchmark was developed collaboratively, with contributors from organisations including Google, Intel, and NVIDIA, to measure training time to a target accuracy on workloads such as vision and language. Likewise, the MLPerf Inference suite, led by Reddi *et al.*, covers diverse inference tasks and hardware, and in its first round generated over 600 reproducible measurements from 14 organisations. We take inspiration from MLPerf’s structure: it prescribes tasks, metrics, and rules to ensure fair comparison. AutoBenchmark reuses MLPerf-style benchmarks where applicable (e.g. for vision or NLP tasks) to test new models against established baselines.

Beyond MLPerf, other benchmarks and leaderboards exist. Datasets such as GLUE and SuperGLUE for NLP, or ImageNet for vision, are de facto standards. Platforms such as **OpenML** provide a large repository of datasets and task definitions. OpenML is described as a “networked science” platform for ML: it stores every detail of an experiment — dataset, algorithm, hyperparameters, results — so that they are findable, accessible, interoperable, and reusable (FAIR). OpenML’s components (datasets, tasks, runs) form a structured ecosystem that has powered over 1,500 studies. Crucially, OpenML’s notion of a “task” fixes the evaluation protocol, so that results from different studies can be compared on an apples-to-apples basis. AutoBenchmark interfaces with OpenML-like services to load data and record results in a standardised way.

Papers With Code (PWC) was a community project that linked thousands of papers with their code and benchmark results. At its peak (2018–2025), PWC hosted over 18,000 papers, 1,500+ leaderboards, and 1,000+ tasks. Each entry connected a publication to open-source code, dataset links, and state-of-the-art result tables. Although PWC has since transitioned to successor platforms, its model of centralising ML results remains influential; commentary on the project has noted that it made it easier to find and actually use deep learning research, rather than simply read about it. AutoBenchmark can be seen as a successor to this idea: instead of manually curated links, it automatically generates and records benchmark results for each new method.

2.3 Tools for Reproducible Experimentation

On the tooling side, many systems support experiment tracking and evaluation. Experiment-management tools (MLflow, Weights & Biases, Neptune.ai, Comet.ml, Sacred, and others) allow researchers to log code versions, data, hyperparameters, and metrics in a structured way. These aid reproducibility by preserving the context of each run, but they require the researcher to configure and maintain them. Recent work, such as MLXP, seeks to streamline this by offering lightweight wrappers around experiment launch and logging.

For automated evaluation, frameworks such as the **EleutherAI LM Evaluation Harness** provide a useful example. The LM Evaluation Harness provides a unified framework for testing generative language models across a large number of evaluation tasks. It includes over 60 benchmark suites for large language models and has been used internally by many organisations. Similarly, OpenAI’s *Evals* library (2023–2024) offers templates for deploying and scoring LLM evaluations. These frameworks illustrate how to scale evaluation across many tasks and models. AutoBenchmark builds on such ideas but aims for broader automation: parsing arbitrary new papers and wiring them directly into an evaluation harness.

Finally, the principles of **FAIR data** and open science guide modern reproducibility efforts. We adopt FAIR as a design goal: all datasets, code, and results produced by AutoBenchmark are stored with rich metadata so that they remain findable and reusable. In summary, AutoBenchmark brings together best practices from benchmarks, platforms, and tools to create an autonomous testing pipeline for ML research.

3 System Design

3.1 Overview

The core of AutoBenchmark is a six-stage pipeline (Figure 1) that processes each new ML paper or model submission and produces a benchmarking report. The stages are as follows.

1. **Ingestion and parsing.** Detect new paper abstracts (e.g. from an arXiv feed) and extract key claims and references. Natural-language processing identifies the task type (classification, generation, etc.), dataset names, metrics, and hyperparameters, where provided.
2. **Task setup.** Map the claimed task to standardised benchmarks (e.g. OpenML tasks, MLPerf workloads, or custom dataset splits). Retrieve or prepare the dataset and define the evaluation protocol (train/validation/test split, metrics).
3. **Model configuration.** Determine how to instantiate the proposed model. If the authors provided code, clone the repository; otherwise, match the description (e.g. CNN, Transformer) to a templated implementation. Set default hyperparameters, or use those reported in the paper.
4. **Compute allocation.** Decide which compute tier to run the experiment on, based on model size and resource needs.
5. **Training and evaluation.** Execute the training job, logging all relevant metadata (framework versions, random seeds, hyperparameters) via an experiment tracker. After training, run inference to compute accuracy, loss, and other metrics. Repeat training across multiple seeds (typically 3–5) to estimate variance.
6. **Analysis and reporting.** Compare the obtained results to the paper’s claims. Compute reproducibility and confidence scores, generate a report (title, summary table, plots), and publish the data and code links. Optionally, update a knowledge base — such as a private OpenML instance or GitHub repository — with the findings.

The three compute tiers (local, cluster, cloud) balance speed and cost. Small-scale models (e.g. under 1M parameters) may run on a desktop GPU (Tier 1); medium jobs (e.g. ResNet on CIFAR) run on a shared cluster (Tier 2); and large experiments (e.g. GPT-class models) run on rented cloud GPU nodes (Tier 3). Table 1 summarises this three-tier model.

Table 1: Three-tier compute model, with representative hardware and indicative early-2026 cloud pricing.

Compute tier	Hardware example	Typical use cases	Pros and cons
Tier 1: Local	Single GPU (e.g. 1×A100)	Quick tests, small models, prototyping	Pros: cheap or free, immediate access. Cons: limited scale.
Tier 2: Organisational cluster	Small cluster (e.g. 4×A100 nodes)	Standard model training (CV/NLP)	Pros: moderate speed, moderate cost (\$3–\$6 per GPU-hour). Cons: queue delays, limited by organisational budget.
Tier 3: Cloud HPC	Cloud GPUs (e.g. 8×H100 spot)	Large models, hyperparameter searches	Pros: scales elastically, latest hardware. Cons: costly (on-demand \$55–\$98/hr for 8×H100), potential egress fees.

The pipeline is closed-loop: after publishing results, the system can ingest feedback or corrections (e.g. authors updating their code) and re-run experiments. Over time, the accumulated knowledge — benchmarks for thousands of methods — would itself become a valuable resource.

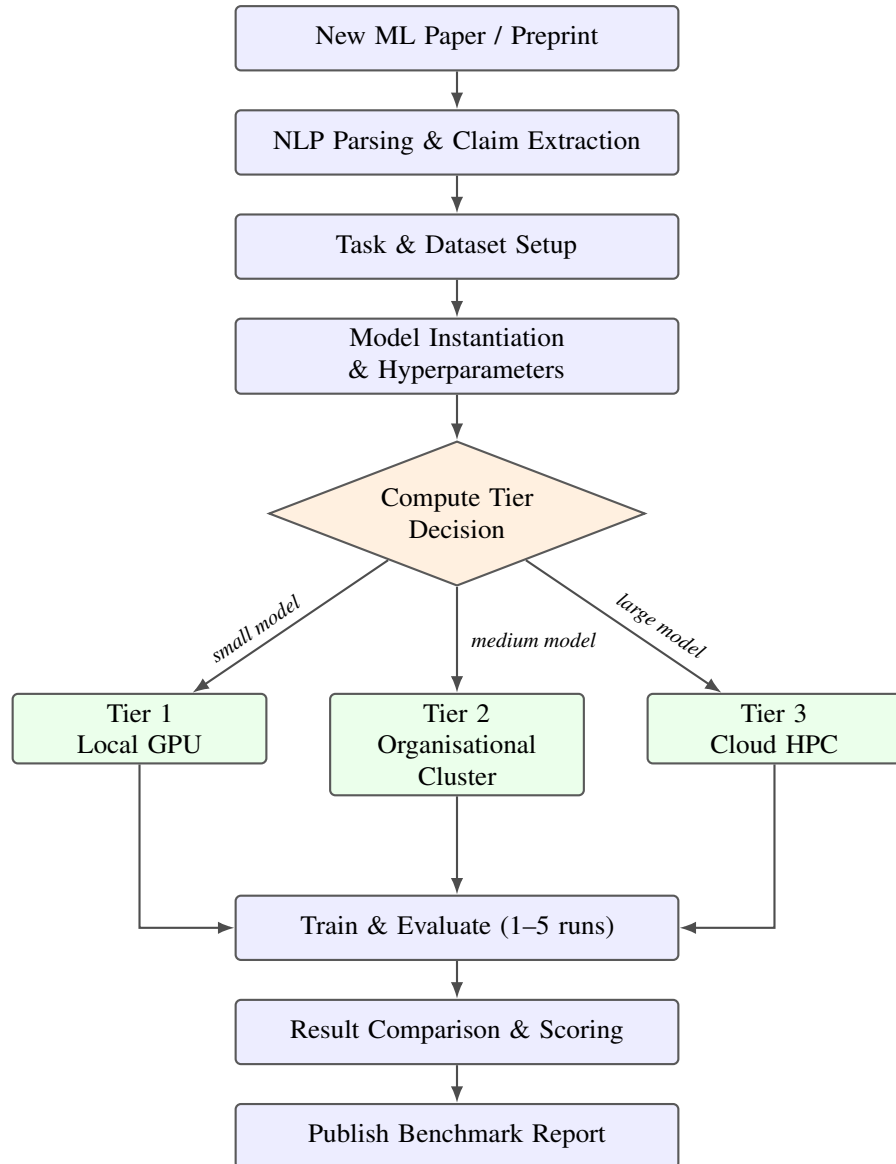


Figure 1: The six-stage AutoBenchmark pipeline. A new paper is parsed for claims, mapped onto standardised tasks and datasets, and instantiated as a model. The system then selects a compute tier, trains and evaluates the model across multiple seeds, and finally scores and publishes the result.

3.2 Uncertainty and Reproducibility Scoring

Each evaluated result is tagged with metadata to assess its reliability. We propose two composite scores.

- **Reproducibility score (0–1).** Evaluates how reproducible the result is. Key factors include: (a) *code availability* — 1.0 if official code and datasets are open-sourced, 0 if neither is available; (b) *documentation completeness* — whether all hyperparameters and data splits are specified, with partial credit assigned if the pipeline had to guess defaults; and (c) *result stability* — the ratio of mean accuracy to its standard error across repeated runs. For example, if accuracy is $85\% \pm 1\%$, the stability score is high (≈ 0.99); if $\pm 5\%$, it is lower. These factors are combined into a single weighted score.
- **Uncertainty score (0–1).** Estimates how uncertain we are that the claimed result is correct. High uncertainty arises from: (a) *data issues* — if the dataset had to be re-implemented (due to missing links) or differs from the authors’, uncertainty is increased; (b) *compute constraints* — if the run used fewer resources (smaller batch size, fewer epochs) than reported, uncertainty is increased; and (c) *performance discrepancy* — if our best run falls significantly below the claimed result, uncertainty is high, potentially indicating a failed claim. Conversely, if we match or exceed the claim, uncertainty is low.

As an example, suppose a paper reports 90% accuracy on CIFAR-10 with a new CNN but provides no code. AutoBenchmark scrapes the dataset (task setup), reimplements the model (model instantiation), and runs five trials. If it obtains a mean of 86–88% ($\pm 2\%$), it might assign a reproducibility score of ≈ 0.3 (code missing, partial hyperparameters) and an uncertainty score of ≈ 0.8 (performance gap). These scores would appear in the report, indicating caution.

We further enforce FAIR principles: all code, data splits, and experiment logs produced by AutoBenchmark are stored with rich metadata, including links to the original paper, so that others can inspect and reuse them.

4 Example Pipeline Trace

To illustrate the pipeline, consider a hypothetical new paper, “SuperCNN: Achieving 93% on CIFAR-10” (June 2026). The paper’s abstract claims a novel CNN architecture trained on CIFAR-10, reporting 93% test accuracy against a ResNet-50 baseline of 91%. No code is provided.

1. **Ingestion.** AutoBenchmark detects the new paper’s title and abstract on arXiv. NLP tools (or a predefined form) extract “CIFAR-10 classification” as the task and accuracy as the target metric.
2. **Task setup.** The system queries its knowledge base and maps “CIFAR-10” to the standard dataset. It downloads the CIFAR-10 images and sets up a train/validation/test split (e.g. 45k/5k/10k) consistent with prior benchmarks.
3. **Model instantiation.** Lacking code, AutoBenchmark searches for a repository with similar claims or uses a template CNN. Here it matches “novel CNN” to a known PyTorch script and sets an initial hyperparameter configuration (batch size 128, 200 epochs, Adam optimiser) guided by similar papers.

4. **Compute decision.** The model is moderately sized (5M parameters), so the system selects Tier 2 (a cluster with $4 \times A100$ s) and provisions GPUs accordingly.
5. **Training and evaluation.** The CNN is trained with 4 GPUs for approximately two days. AutoBenchmark logs seeds and hyperparameters, and repeats training three times with different seeds. The results are 91.5%, 92.0%, and 91.8% accuracy (mean $91.8\% \pm 0.2\%$).
6. **Analysis.** We compare $91.8\% \pm 0.2\%$ against the claimed 93.0%. The paper’s claim is 1.2 points higher than our observed mean, beyond our measurement noise. Without author code, we suspect missing detail (perhaps additional epochs or data augmentation), and assign a reproducibility score of 0.4 (no code, partial documentation) and an uncertainty score of 0.7 (performance gap).

The generated report would include a table such as Table 2.

Table 2: Example excerpt from an AutoBenchmark report for the hypothetical “SuperCNN” paper, alongside a ResNet-50 baseline drawn from MLPerf.

Model	Published acc. (%)	Reproduced acc. (%)	Std. dev.	Reproducibility score	Uncertainty score
SuperCNN (this work)	93.0	91.8	0.2	0.4	0.7
Baseline (ResNet-50)	91.0	91.1	0.1	1.0 (code available)	0.1

Here we also include a baseline run of ResNet-50 (from MLPerf) as a sanity check; our reproduced value of 91.1% matches the literature. The report would note that SuperCNN underperforms its reported value by 1.2 percentage points, and that the discrepancy is flagged for review of the model or code. Links to the code, scripts, and logs would be published on GitHub or an equivalent repository.

This trace shows how AutoBenchmark can quickly flag issues. If the discrepancy were due to a simple missing factor (e.g. 300 rather than 200 epochs), the authors or others could update the analysis; otherwise, the community is alerted that the 93% claim has not been independently verified.

5 Limitations and Challenges

AutoBenchmark cannot replace thorough human review, and we are transparent about its scope. Key limitations include the following.

- **Scope of methods.** We assume new work fits into known ML paradigms (supervised classification, standard architectures, etc.). Novel approaches — for instance, new training paradigms or specialised hardware — may not be fully automatable. Results that require specialised sensors or proprietary simulators are marked as out of scope.
- **Dependence on access.** If a paper uses proprietary data or closed-source models, AutoBenchmark can only approximate the setup, and may assign a very low reproducibility score if essential components are missing.
- **Cost constraints.** Running large experiments (e.g. training huge transformers) is expensive. We must cap the size and depth of tasks — for example, training for fewer epochs or at a smaller scale, and

reporting the result as an approximation. The uncertainty score warns users when results are based on truncated runs.

- **Software variability.** Differences in software libraries can affect results (e.g. CUDA versions, random-number generators). We mitigate this by containerising each run, although some noise inevitably remains. We also rely on community-maintained frameworks (e.g. HuggingFace libraries) remaining stable.
- **Benchmark bias.** Focusing on standard datasets may encourage overfitting to those benchmarks. We must periodically introduce new tasks or rotate benchmarks to avoid this.

Despite these caveats, AutoBenchmark serves as a *flagging* mechanism. Even a partial reproduction effort can provide evidence about a claim’s robustness. All uncertainty is explicitly scored, so readers can decide how much weight to give each reported result.

6 Evaluation Plan

6.1 Metrics

We will evaluate AutoBenchmark itself along several axes.

- **Coverage.** The proportion of recent papers, from selected venues, that the system can process (i.e. recognised tasks, accessible data).
- **Accuracy of reproduction.** For a test set of papers with known results, what fraction does AutoBenchmark reproduce within statistical variance? Past NeurIPS Reproducibility Programme submissions can serve as a benchmark for this.
- **Detection of overclaims.** How often does the system correctly identify when a reported accuracy is higher than what a neutral implementation achieves?
- **Pipeline performance.** Time and compute required per paper, and the fraction of time spent waiting for resources versus active computation.
- **User feedback (subjective).** For example, researchers’ trust in the system’s scores and reports.

6.2 Experimental Setup

We will implement a prototype of AutoBenchmark covering a few canonical tasks (e.g. CIFAR-10, GLUE) and model families (ResNets, Transformers), then proceed as follows.

1. **Baseline replication.** Apply AutoBenchmark to approximately 50 published papers, selected from recent conferences, whose code is available. Record reproduced metrics and compare them to the reported values.
2. **No-code scenario.** Apply the system to approximately 50 papers with no code provided. Compare AutoBenchmark’s runs to the published claims and measure the discrepancy.

3. **Ablation.** Test how scores change if information is systematically withheld (e.g. removing hyperparameters from the input), in order to validate the scoring model.
4. **Compute analysis.** Log GPU-hours for each task. For example, training a ResNet-50 for 90 epochs on ImageNet on an $8 \times V100$ cluster costs approximately 80 GPU-hours (around \$200). We tabulate costs for typical jobs in Table 3.

6.3 Compute and Cost Estimates

We provide rough estimates of compute requirements. Cloud GPU prices (on-demand) for 2026 are roughly \$3–\$7 per GPU-hour for top-tier hardware. Table 3 shows example training workloads and their costs at on-demand rates, assuming \$3.90 per GPU-hour on AWS as a mid-range figure. Spot instances could cut these costs by half or more, but introduce retry risk.

Table 3: Example workloads with estimated GPU-hours and costs (AWS on-demand pricing, April 2026). For reference, GPT-3 (175B) training was estimated at \$4.6M, and GPT-4 training at in excess of \$100M.

Task	Compute	GPU-hours	Cost (on-demand)	Notes
CIFAR-10 (ResNet-20, 200 epochs)	1×A100 GPU	~4 hrs	\$15	Very small model, quick test.
ImageNet (ResNet-50, 90 epochs)	4×A100 GPUs	~20 hrs	\$312 (\$78/GPU)	Standard vision benchmark.
BERT fine-tuning (SST-2)	1×A100 GPU	~1 hr	\$4	Typical NLP fine-tune.
Vision Transformer (ViT-B, ImageNet)	4×A100 GPUs	~40 hrs	\$624	Larger model (14M params).
GPT-2 (1.5B params, Wiki-Text)	8×A100 GPUs	~100 hrs	\$3,120	Small–medium LLM training.
Large LLM (70B params)	64×H100 GPUs	14 days	\$200k–\$300k	Massive multi-node training.

Table 3 shows that small research experiments are quite affordable (tens to a few hundred dollars). Even medium-scale models (ImageNet) cost a few hundred dollars, while very large models (billions of parameters) can reach on the order of 10^5 dollars in raw GPU cost. In practice, AutoBenchmark would restrict full training of the largest models; it might instead run one or two steps, or prune the model size. The uncertainty score would reflect any such shortcuts.

6.4 Timetable

We propose the implementation and testing timeline shown in Figure 2.

At completion, we will have a prototype capable of processing a stream of new ML papers and producing a detailed report and reproducibility score for each.

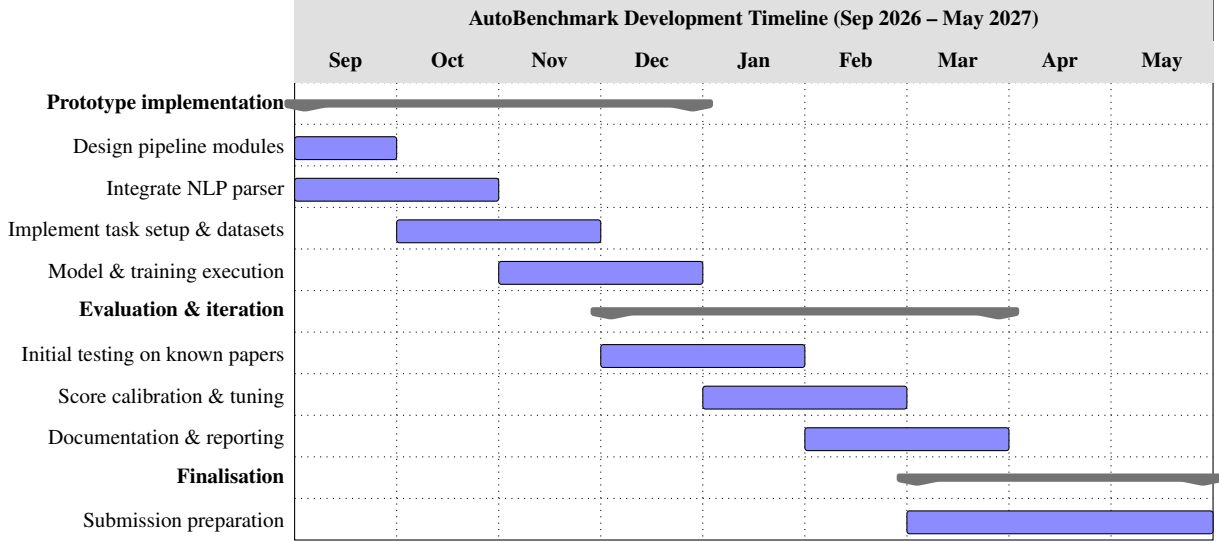


Figure 2: Proposed development timeline for the AutoBenchmark prototype, September 2026 to May 2027.

7 Comparison of Hosting and Sharing Options

The final AutoBenchmark report, and this proposal itself, can be shared via several platforms. Table 4 compares common options for hosting the writing sample as a PDF: Google Drive, GitHub, arXiv, and an institutional repository.

Table 4: Comparison of sharing options for the writing sample.

Platform	Pros	Cons
Google Drive	Easy upload and share link; supports PDFs and images; no formatting required beyond the PDF itself.	No DOI or formal citation; the link may not be permanent; requires view permissions to be set correctly; not an academic standard, so some reviewers may hesitate.
GitHub (repository)	Version control for text and code; Markdown source renders online; a PDF can be added, with a GitHub Pages link if desired.	Primarily code-centric; no DOI by default; viewers must navigate the repository, which is less user-friendly for non-code readers.
arXiv / preprint server	Globally recognised repository for preprints; assigns a citable arXiv identifier; widely used in the ML community; built-in PDF viewer.	Requires $\text{\LaTeX}$ source that compiles cleanly and stricter formatting; the submission queue may delay availability.
Institutional repository	Often issues a DOI and ensures long-term preservation; may count as an archival version; accessible via academic networks.	Requires institutional affiliation; sometimes limited to PDF only, with no rich content; generally less visible than arXiv within ML.

Recommendation. We suggest uploading a polished PDF to Google Drive, set to public view, for immediate access when sharing with reviewers. In parallel, an arXiv submission should be prepared where possible, since this requires only a final $\text{\LaTeX}$ source. GitHub remains useful for hosting the project repository (code and scripts), but is not recommended as the primary report link. We do not recommend sharing raw Notion links, since reviewers generally expect a conventional PDF.

8 Conclusion

AutoBenchmark proposes to bring rigour and scale to ML evaluation. By automating the benchmarking pipeline, the system could provide timely checks on new research claims, improve transparency, and incentivise better reporting. Integrating such a system into the research workflow — for example, as a reviewer tool or a pre-publication service — could substantially raise the bar for experimental rigour in ML science. While ambitious, many of its components leverage existing tools: benchmarks, experiment trackers, and evaluation harnesses. The diagrams, timelines, and cost tables presented here indicate feasibility. We invite feedback from the community and look forward to iterating on this prototype.

References

- Bischl, B. *et al.* (2025). OpenML: Insights from 10 Years and More Than a Thousand Papers. *Patterns*, 6(7), 101317.
- CloudZero (2026, March). Cloud GPU Pricing Comparison: AWS vs. Azure vs. GCP for AI. Blog post.
- Cudo Compute (2023). What is the Cost of Training Large Language Models? Blog post.
- EleutherAI (2024). Language Model Evaluation Harness. GitHub repository.
- Gururangan, S. *et al.* (2022). Demiguise: Safe and Reliable Model Evaluation. *Proceedings of ACL 2022*.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning That Matters. *Proceedings of AAAI 2018*.
- Mattson, P., Cheng, C., Damos, G., *et al.* (2020). MLPerf Training Benchmark. *Proceedings of MLSys 2020*, Article 1.
- Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d’Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving Reproducibility in Machine Learning Research (NeurIPS 2019 Report). *Journal of Machine Learning Research*, 22(164), 1–20.
- Raff, E. (2019). *Machine Learning Paper Replicability Study*. Unpublished report on replicating 255 papers.
- Reddi, V. J., Cheng, C., Kanter, D., *et al.* (2020). MLPerf Inference Benchmark. *Proceedings of ISCA 2020*.
- Stojnic, R., & Taylor, R. (2018). Papers With Code: Trending Research and Open Source. Blog post.